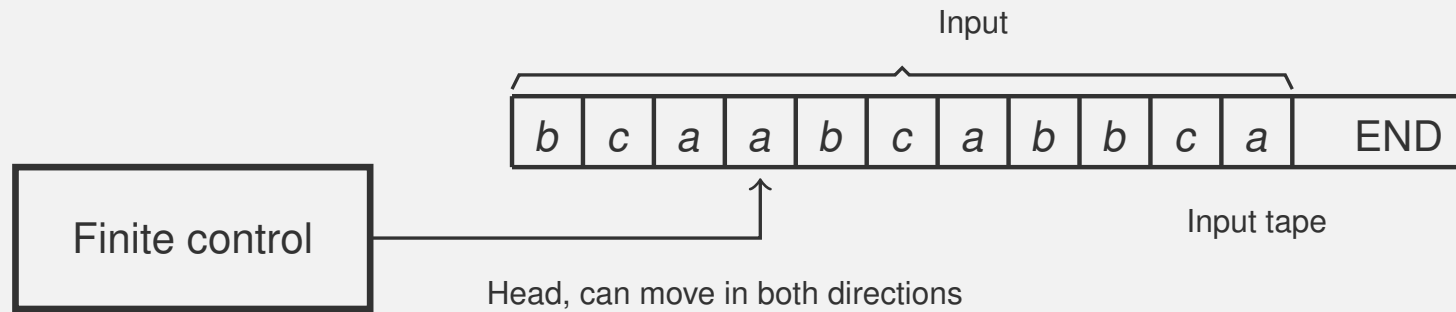


Fundamentals of theory of computation I.

Lecture 4

Krisztián Tichler
ktichler@inf.elte.hu

Finite automata



- ▶ Formal languages can be given not just by generative tools, but also by recognizing them by mathematical machines, like finite automata.
- ▶ While grammars are synthetic tools, automata are analytic tools.
- ▶ Automata give a binary output once the input is fully processed. An input is either rejected ('no' answer) or accepted ('yes' answer).
- ▶ Finite automata take discrete steps for their inputs. Deterministic automata take as many steps as the length of the input.

Finite automata

- ▶ A finite automaton can be viewed as follows. It has finitely many states and starts from its initial state. An input word is given on an input tape, a tape of discrete cells. Inputs occupy consecutive cells, one letter in each cell, starting from the 1st cell. Initially, the head of the automaton is on the first cell.
- ▶ Input words are processed letter by letter. Once the automaton reads a letter from the input, the head moves one cell to the right. Next state is determined according to its transition function.
- ▶ If the input is not yet fully processed the automaton continues working according to its rules. So, reaching an accepting state while there are still letters to be processed is not enough for accepting the input.
- ▶ To accept an input the automaton should process all the letters and the state at halting should be an accepting state.

Deterministic finite automaton

Definition

A **deterministic finite automaton** is a quintuple,

$A = (Q, \Sigma, \delta, q_0, F)$, where

- ▶ Q is a finite, nonempty set, the **set of states**,
- ▶ Σ is the alphabet of the input symbols (**input alphabet**),
- ▶ δ is the **(state) transition function**, a total mapping from the set $Q \times \Sigma$ to Q (the domain of δ is $Q \times \Sigma$),
- ▶ $q_0 \in Q$ is the **initial state** (or start state),
- ▶ $F \subseteq Q$ is the **set of accept(ing) states**.

So, for all $(q, a) \in Q \times \Sigma$ there exists **exactly one** state s , such that $\delta(q, a) = s$ holds.

Extended transition function

Let $A = (Q, \Sigma, \delta, q_0, F)$ be a deterministic finite automaton (DFA).

Definition

The **extended transition function** is a (total) mapping

$\hat{\delta} : Q \times \Sigma^* \rightarrow Q$, where

- ▶ $\hat{\delta}(q, \varepsilon) := q$
- ▶ $\hat{\delta}(q, xa) := \delta(\hat{\delta}(q, x), a) \quad (\forall x \in \Sigma^*, a \in \Sigma)$

So, $\hat{\delta}(q, u)$ is the state where the DFA halts when the word u is fully processed starting from state q .

So u is accepted if $\hat{\delta}(q_0, u) \in F$ and u is rejected if $\hat{\delta}(q_0, u) \notin F$.

Observe, that $\hat{\delta}$ is an extension of δ (they give the same results for words of length 1). Therefore, we omit the $\hat{\delta}$ on δ in most of the future uses.

Deterministic finite automaton – example

Example 1. Let $A = (Q, \Sigma, \delta, q_0, F)$ be a finite automaton, where $Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{a, b\}$, $F = \{q_1, q_2\}$ and let

$$\begin{array}{ll} \delta(q_0, a) = q_2, & \delta(q_0, b) = q_1, \\ \delta(q_1, a) = q_3, & \delta(q_1, b) = q_0, \\ \delta(q_2, a) = q_0, & \delta(q_2, b) = q_3, \\ \delta(q_3, a) = q_1, & \delta(q_3, b) = q_2. \end{array}$$

E.g.,

$$\delta(q_2, abb) = \delta(\delta(\delta(q_2, a), b), b) = \delta(\delta(q_0, b), b) = \delta(q_1, b) = q_0.$$

q_0 : even number of a 's and even number of b 's so far.

q_1 : even number of a 's and odd number of b 's so far.

q_2 : odd number of a 's and even number of b 's so far.

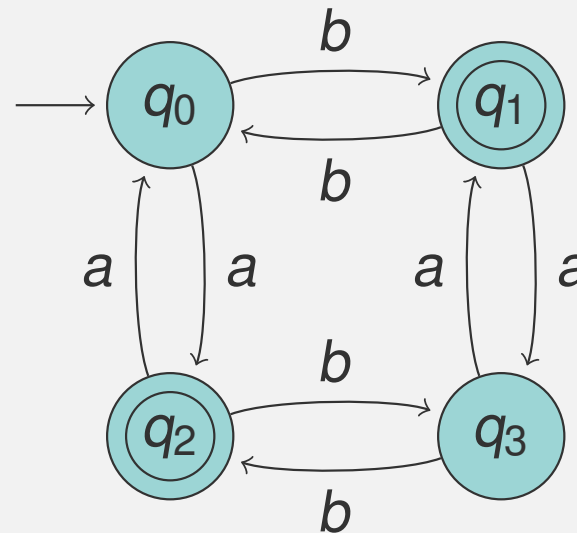
q_3 : odd number of a 's and odd number of b 's so far.

A accepts exactly those words $u \in \{a, b\}^*$, such that $N_a(u)$ and $N_b(u)$ has different parity (even-odd or odd-even).

Deterministic finite automaton – example

1. Finite automata can also be given by a **(state) transition table** of size $|Q| \times |\Sigma|$ (content of the cell in row $q \in Q$ and column $t \in \Sigma$ is $\delta(q, t)$, $\rightarrow$: initial state, $\leftarrow$: accept state, $\Leftrightarrow$: both initial and accept state).

	a	b
$\rightarrow q_0$	q_2	q_1
$\leftarrow q_1$	q_3	q_0
$\leftarrow q_2$	q_0	q_3
q_3	q_1	q_2



2. Or by a **(state) transition diagram** (labelled, directed graph of size $|Q|$, nodes are labelled by the elements of Q , there is an edge from $q \in Q$ to $r \in Q$ with the label $t \in \Sigma$ if and only if $\delta(q, t) = r$ holds. Start state is denoted by $\rightarrow$, accept states are double circled.)

DFA vs NFA

Deterministic finite automaton:

- ▶ δ is one-valued. There exists exactly one state $s \in Q$ with the property $\delta(q, a) = s$ for every $(q, a) \in Q \times \Sigma$.

Nondeterministic finite automaton:

- ▶ δ is multi-valued, i.e., δ is a mapping $Q \times \Sigma \rightarrow \mathcal{P}(Q)$.
- ▶ Multiple start states are possible (we have a set of initial states $Q_0 \subseteq Q$).
- ▶ We may interpret multi-valued transitions as all states of $\delta(q, a)$ are allowed to be the next state for $q \in Q$ and $a \in \Sigma$. So, multiple computations are possible for the same input.
- ▶ As $\emptyset \in \mathcal{P}(Q)$, it is possible to have $\delta(q, a) = \emptyset$ for some $(q, a) \in Q \times \Sigma$. In this case, the machine gets stuck.
- ▶ It accepts an input if and only if the NFA has a complete computation (no letter left to be processed) ending in a state from F .

Nondeterministic finite automaton

Definition

A **nondeterministic finite automaton** is a quintuple, $A = (Q, \Sigma, \delta, q_0, F)$, where

- ▶ Q is a finite, nonempty set, the **set of states**,
- ▶ Σ is the alphabet of the input symbols (**input alphabet**),
- ▶ $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ is the **(state) transition function**,
- ▶ $Q_0 \subseteq Q$ is the **set of initial states** (or start states),
- ▶ $F \subseteq Q$ is the **set of accept(ing) states**.

Remark: Deterministic finite automata can be considered as special case of a nondeterministic finite automata. If $|\delta(q, a)| = 1$ holds for every $(q, a) \in Q \times \Sigma$ then the NFA has exactly one computation for all inputs, and works exactly as the corresponding DFA.

Nondeterministic finite automaton – example

Example 2. Let $A = (Q, \Sigma, \delta, Q_0, F)$ be a nondeterministic finite automaton (NFA), where $Q = \{q_0, q_1, q_2, q_3\}$, $\Sigma = \{a, b\}$, $Q_0 = \{q_0, q_2\}$, $F = \{q_1, q_2\}$ and let

$$\delta(q_0, a) = \{q_1, q_2\},$$

$$\delta(q_1, a) = \{q_3\},$$

$$\delta(q_2, a) = \{q_0\},$$

$$\delta(q_3, a) = \{q_1\},$$

$$\delta(q_0, b) = \{q_1\},$$

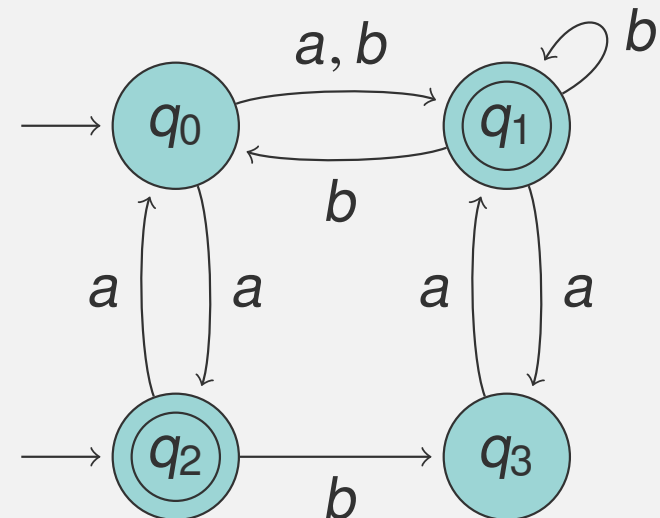
$$\delta(q_1, b) = \{q_0, q_1\},$$

$$\delta(q_2, b) = \{q_3\},$$

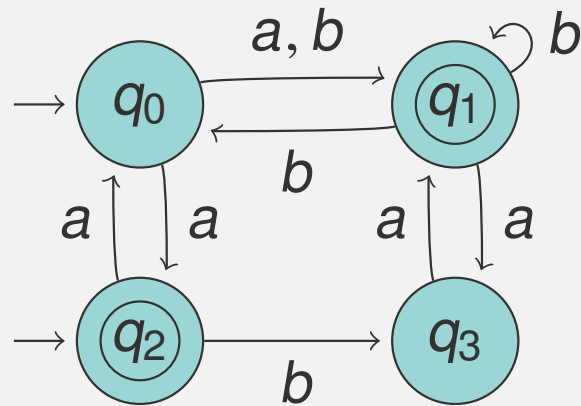
$$\delta(q_3, b) = \{\}.$$

The same automaton by transition table and transition diagram:

	a	b
$\rightarrow q_0$	$\{q_1, q_2\}$	$\{q_1\}$
$\leftarrow q_1$	$\{q_3\}$	$\{q_0, q_1\}$
$\rightleftarrows q_2$	$\{q_0\}$	$\{q_3\}$
q_3	$\{q_1\}$	$\{\}$



Nondeterministic finite automaton – example



Accepts aab , since there exists **at least one** accepting computation.

Some possible computations for input aab .

Case 1.

state	q_2	q_0	q_1	q_0	$\notin F$
to be processed	aab	ab	b	ε	rejecting computation

Case 2.

state	q_0	q_1	q_3	
to be processed	aab	ab	b	computation gets stuck

Case 3.

state	q_2	q_0	q_1	q_1	$\in F$
to be processed	aab	ab	b	ε	accepting computation

An alternative, rule based view

Let $A = (Q, \Sigma, \delta, Q_0, F)$ be a nondeterministic finite automaton.

Transitions viewed as rewriting rules.

For every $p, q \in Q$, $a \in \Sigma$, $p \in \delta(q, a)$ of nondeterministic finite automaton A add the the rewriting rule

$$qa \rightarrow p$$

to the set M_δ , i.e.,

$$M_\delta = \{qa \rightarrow p \mid p, q \in Q, a \in \Sigma, p \in \delta(q, a)\}$$

Observation: A finite automaton A is deterministic if and only if it has a single initial state and for every $(q, a) \in Q \times \Sigma$ there exists exactly one $p \in Q$ with the property $qa \rightarrow p \in M_\delta$.

An alternative, rule based view

Reconsider Example 2, and give the set M_δ .

Example:

$M :$	a	b
$\rightarrow q_0$	$\{q_1, q_2\}$	$\{q_1\}$
$\leftarrow q_1$	$\{q_3\}$	$\{q_0, q_1\}$
$\rightleftharpoons q_2$	$\{q_0\}$	$\{q_3\}$
q_3	$\{q_1\}$	$\{\}$

$M_\delta :$	
$q_0 a \rightarrow q_1 \mid q_2$	$q_0 b \rightarrow q_1$
$q_1 a \rightarrow q_3$	$q_1 b \rightarrow q_0 \mid q_1$
$q_2 a \rightarrow q_0$	$q_2 b \rightarrow q_3$
$q_3 a \rightarrow q_1$	

Remark: Although M_δ is a set of rewriting rules, it is NOT a grammar.

Finite automata – one-step reduction

Observe, that the remaining parts of possible computations of a finite automaton being in state $q \in Q$ depends only on the unprocessed suffix $w \in \Sigma^*$ of the input. This observation motivates the following.

Definition

A word $u \in Q\Sigma^*$ is called a **configuration** of an NFA $A = (Q, \Sigma, \delta, Q_0, F)$.

Definition

Let $A = (Q, \Sigma, \delta, Q_0, F)$ be a finite automaton and let $u, v \in Q\Sigma^*$ be configurations. A **reduces** u to v **in one step** (notation: $u \Rightarrow_A v$), if there exists a rule $qa \rightarrow p \in M_\delta$ (i.e., $p \in \delta(q, a)$) and a word $w \in \Sigma^*$ such that $u = qaw$ and $v = pw$ hold.

Remark: In the case of a deterministic finite automaton there exists a unique $v \in Q\Sigma^*$ with the property $u \Rightarrow_A v$ for every $u \in Q\Sigma^+$.

Finite automata – reduction

Definition

Finite automaton $A = (Q, \Sigma, \delta, Q_0, F)$ **reduces** $u \in Q\Sigma^*$ to $v \in Q\Sigma^*$ (notation: $u \Rightarrow_A^* v$) if either $u = v$ or there is an integer $k \geq 1$ and configurations $w_0, \dots, w_k$ such that $w_{i-1} \Rightarrow w_i$ ($1 \leq i \leq k$), $w_0 = u$ and $w_k = v$ hold.

Remark: An alternative way of defining reduction is recursion on the difference of the lengths of configurations ($|u| - |v|$):

$u \Rightarrow_A^* v$ if and only if either $u = v$ or there is a $z \in Q\Sigma^*$, $|z| = |v| + 1$ with the properties $u \Rightarrow_A^* z$ and $z \Rightarrow_A v$.

Remark: $\Rightarrow_A^*$ is the reflexive, transitive closure of $\Rightarrow_A$.

Remark: $\Rightarrow_A^*$ is basically the same as derivation in the theory of grammars (between configurations by the rules of M_δ).

Remark: In the case of an NFA there may be more than 1 or 0 reductions of a given length from a given configuration. On the other hand, there exists exactly one for DFA's.

Finite automata – Recognized language

Definition

The language **recognized** by a nondeterministic finite automaton $A = (Q, \Sigma, \delta, Q_0, F)$ is

$$L(A) = \{u \in \Sigma^* \mid q_0 u \Rightarrow_A^* p \text{ holds for some } q_0 \in Q_0 \text{ and } p \in F\}$$

Remark: $L(A)$ is also called the language accepted by A .

Remark: $\varepsilon \in L(A) \iff Q_0 \cap F \neq \emptyset$.

Remark: In the case of a DFA we have a unique initial state and for every $u \in \Sigma^*$ we have a unique reduction of $q_0 u$ to some $p \in Q$. If $p \in F$, then u is accepted by A , otherwise u is rejected. I.e.,

$$L(A) = \{u \in \Sigma^* \mid q_0 u \Rightarrow_A^* p \text{ holds for some } p \in F\}$$

Remark: In the case of a DFA we might have an alternative definition for $L(A)$ using the extension of δ as follows.

$$L(A) = \{u \in \Sigma^* \mid \delta(q_0, u) \in F\}.$$

Remark: The index of $\Rightarrow_A$ can be omitted if A is clear from the context.

Finite automata – examples

In **example 1** (DFA) M_δ consists of the following rules.

$$q_0 a \rightarrow q_2, \quad q_0 b \rightarrow q_1, \quad q_1 a \rightarrow q_3, \quad q_1 b \rightarrow q_0$$

$$q_2 a \rightarrow q_0, \quad q_2 b \rightarrow q_3, \quad q_3 a \rightarrow q_1, \quad q_3 b \rightarrow q_2.$$

$$q_0 aab \Rightarrow q_2 ab \Rightarrow q_0 b \Rightarrow q_1.$$

This is the only computation for aab till it is fully processed. Since $q_1 \in F$ holds, we have $aab \in L(A)$.

In **example 2** (NFA) M_δ consists of the following rules.

$$q_0 a \rightarrow q_1 \mid q_2, \quad q_0 b \rightarrow q_1, \quad q_1 a \rightarrow q_3, \quad q_1 b \rightarrow q_0 \mid q_1$$

$$q_2 a \rightarrow q_0, \quad q_2 b \rightarrow q_3, \quad q_3 a \rightarrow q_1.$$

Consider the following reductions.

$$q_2 aab \Rightarrow q_0 ab \Rightarrow q_1 b \Rightarrow q_0. \quad \text{(it was case 1 earlier)}$$

$$q_2 aab \Rightarrow q_0 ab \Rightarrow q_1 b \Rightarrow q_1. \quad \text{(it was case 3 earlier)}$$

These are 2 different computation for aab till it is fully processed.

Since one accepting computation is enough and $q_1 \in F$ holds, we have $aab \in L(A)$ (2nd computation).

Finite automata and regular languages

Theorem

For every nondeterministic finite automaton A there is a type 3 grammar G with the property $L(G) = L(A)$.

Proof: Let $A = (Q, \Sigma, \delta, Q_0, F)$ be a nondeterministic finite automaton.

Every transition rule $qa \rightarrow p \in M_\delta$ ($q, p \in Q, a \in \Sigma, p \in \delta(q, a)$) is length-decreasing, but one might observe, that every reduction corresponds to a derivation in a left-linear grammar as follows.

Let $G = (N, \Sigma, P, S)$ be a grammar, where $N = Q \cup \{S\}$ ($S \notin Q$) and P is defined as follows.

1. $p \rightarrow a : \in P$ if and only if $q_0 a \rightarrow p \in M_\delta$ holds for some $q_0 \in Q_0$,
2. $p \rightarrow qa : \in P$ if and only if $qa \rightarrow p \in M_\delta$,
3. $S \rightarrow p : \in P$ if and only if $p \in F$,
4. $S \rightarrow \varepsilon : \in P$ if and only if $Q_0 \cap F \neq \emptyset$

Finite automata and regular languages

Obviously $\varepsilon \in L(G)$ if and only if $\varepsilon \in L(A)$.

$L(A) \subseteq L(G)$:

- ▶ Assume, that $u \neq \varepsilon$, $u \in L(A)$. Then, by definition, there is a $q_0 \in Q_0$ and $p \in F$ with $q_0 u \Rightarrow_A^* p$. Consider the reversal of this reduction. According to our construction we have the derivation $p \Rightarrow_G^* q_0 u$ (by the rules of 2.).
- ▶ The last step of this derivation in G is applying a rule $p_1 \rightarrow q_0 a$ for some $p_1 \in Q$. Then we have $q_0 a \rightarrow p_1 \in M_\delta$ (by 2.), but then $p_1 \rightarrow a \in P$ (by 1.). So, we have $p \Rightarrow_G^* u$ if we apply $p_1 \rightarrow a$ instead of $p_1 \rightarrow q_0 a$ in the last step.
- ▶ Since we have $p \in F$, then we also have $S \rightarrow p \in P$ (by 3.) implying $S \Rightarrow_G^* u$.
- ▶ Since u is an arbitrary, nonempty word of $L(A)$, we have $L(A) \subseteq L(G)$.

Finite automata and regular languages

$L(G) \subseteq L(A)$:

- ▶ Let $u \in L(G)$ and $u \neq \varepsilon$. Then we have $S \Rightarrow_G^* u$.
- ▶ According to the rules of G this derivation should be of the form

$$S \Rightarrow_G p \Rightarrow_G^* p_1 v \Rightarrow_G av = u,$$

where $p \in F$ and $p_1 \rightarrow a \in P$.

- ▶ Since $p_1 \rightarrow a \in P$ holds, we have $q_0 a \rightarrow p_1 \in M_\delta$ for some $q_0 \in Q_0$ (by 1.). Then we have $p_1 \rightarrow q_0 a \in P$ (by 2.). Exchanging the last rule $p_1 \rightarrow a$ applied in the derivation to $p_1 \rightarrow q_0 a$ we have $p \Rightarrow_G^* q_0 u$.
- ▶ Then $q_0 u \Rightarrow_A^* p$ holds by 2..
- ▶ Since $p \in F$, we have $u \in L(A)$, implying $L(G) \subseteq L(A)$.

Grammar G is left-linear. According to a previous theorem there is a right-linear grammar G' with the property $L(G') = L(G)$ proving the theorem. □

Finite automata and regular languages

Corollary:

NFA		NFA (M_δ)	left-linear	right-linear
initial state q_0			NEW initial symbol S	
$p \in \delta(q_0, a)$		$q_0 a \rightarrow p \ (q_0 \in Q_0)$	$p \rightarrow a$	$S \rightarrow ap$
$p \in \delta(q, a)$		$qa \rightarrow p$	$p \rightarrow qa$	$q \rightarrow ap$
$p \in F$			$S \rightarrow p$	$p \rightarrow \varepsilon$
$Q_0 \cap F \neq \emptyset$			$S \rightarrow \varepsilon$	$S \rightarrow \varepsilon$

Corollary: (if A is deterministic)

DFA	DFA (M_δ)	right-linear
initial state q_0		initial symbol q_0
$\delta(q, a) = p$	$qa \rightarrow p$	$q \rightarrow ap$
$p \in F$		$p \rightarrow \varepsilon$

Finite automata and regular languages

Example 1.

	a	b
$\rightarrow q_0$	q_2	q_1
$\leftarrow q_1$	q_3	q_0
$\leftarrow q_2$	q_0	q_3
q_3	q_1	q_2

Type 3 grammar:

$$q_0 \rightarrow aq_2 \mid bq_1$$

$$q_1 \rightarrow aq_3 \mid bq_0 \mid \varepsilon$$

$$q_2 \rightarrow aq_0 \mid bq_3 \mid \varepsilon$$

$$q_3 \rightarrow aq_1 \mid bq_2$$

Example 2.

	a	b
$\rightarrow q_0$	$\{q_1, q_2\}$	$\{q_1\}$
$\leftarrow q_1$	$\{q_3\}$	$\{q_0, q_1\}$
$\Leftrightarrow q_2$	$\{q_0\}$	$\{q_3\}$
q_3	$\{q_1\}$	$\{\}$

Type 3 grammar:

$$S \rightarrow aq_1 \mid aq_2 \mid bq_1 \mid aq_0 \mid bq_3 \mid \varepsilon$$

$$q_0 \rightarrow aq_1 \mid aq_2 \mid bq_1$$

$$q_1 \rightarrow aq_3 \mid bq_0 \mid bq_1 \mid \varepsilon$$

$$q_2 \rightarrow aq_0 \mid bq_3 \mid \varepsilon$$

$$q_3 \rightarrow aq_1$$

Finite automata and regular languages

Theorem

For every type 3 grammar G there is a nondeterministic finite automaton A with the property $L(A) = L(G)$.

Proof:

- ▶ We can assume, that $G = (N, \Sigma, P, S)$ is in type 3 normal form (all rules are either of the form $X \rightarrow aY$ or $X \rightarrow \varepsilon$, where $X, Y \in N$ and $a \in \Sigma$).
- ▶ Let $A = (Q, \Sigma, \delta, Q_0, F)$ be the nondeterministic finite automaton, where

$$Q = N, \quad Q_0 = \{S\}, \quad \text{and} \quad F = \{Z \in N \mid Z \rightarrow \varepsilon \in P\}.$$

M_δ is defined as follows.

$$Xa \rightarrow Y \in M_\delta \text{ if and only if } X \rightarrow aY \in P.$$

Finite automata and regular languages

- ▶ It is easy to see by induction on the length of derivation/reduction, that

$$X \Rightarrow_G^* uY \iff Xu \Rightarrow_A^* Y$$

holds for every $X, Y \in N, u \in \Sigma^*$.

- ▶ Consider a derivation $S \Rightarrow_G^* uZ \Rightarrow_G u$ in G . So, there is a reduction $Su \Rightarrow_A^* Z$ in A . The last rule applied should have been $Z \rightarrow \varepsilon$. So we have $Z \in F$, be the construction.
- ▶ If $Su \Rightarrow_A^* Z, Z \in F$ holds, then there is a derivation $S \Rightarrow_G^* uZ$ in G . Since $Z \in F$ we have a rule $Z \rightarrow \varepsilon$ by construction, implying $S \Rightarrow_G^* uZ \Rightarrow u$. □

Example:

$S \rightarrow aA \mid aB \mid bB \mid \varepsilon$

$A \rightarrow bS \mid bB$

$B \rightarrow aS \mid aA \mid \varepsilon$

	a	b
$\hookrightarrow S$	$\{A, B\}$	$\{B\}$
A	$\{\}$	$\{S, B\}$
$\leftarrow B$	$\{S, A\}$	$\{\}$

NFA to DFA conversion

Theorem

For every nondeterministic finite automaton $A = (Q, \Sigma, \delta, Q_0, F)$ there is a deterministic finite automaton $A' = (Q', \Sigma, \delta', q'_0, F')$ with the property $L(A') = L(A)$.

Construction:

$$Q' := \mathcal{P}(Q), \quad q'_0 := Q_0, \quad F' := \{q' \in Q' \mid q' \cap F \neq \emptyset\},$$

$$\delta'(q', a) := \bigcup_{q \in q'} \delta(q, a).$$

The theorem is to be proved on the next class.

According to this theorem, both DFA's and NFA's are formal concepts describing exactly the class of regular languages ($\mathcal{L}_3$).

NFA to DFA conversion – example

	<i>a</i>	<i>b</i>
$\rightarrow q_0$	$\{\}$	$\{q_1, q_2\}$
$\Leftrightarrow q_1$	$\{q_0\}$	$\{\}$
$\leftarrow q_2$	$\{q_1\}$	$\{q_2\}$

NFA

$\{\}$
 $\{q_0\}$
 $\leftarrow \{q_1\}$
 $\leftarrow \{q_2\}$
 $\Leftrightarrow \{q_0, q_1\}$
 $\leftarrow \{q_0, q_2\}$
 $\leftarrow \{q_1, q_2\}$
 $\leftarrow \{q_0, q_1, q_2\}$

<i>a</i>	<i>b</i>
$\{\}$	$\{\}$
$\{\}$	$\{q_1, q_2\}$
$\{q_0\}$	$\{\}$
$\{q_1\}$	$\{q_2\}$
$\{q_0\}$	$\{q_1, q_2\}$
$\{q_1\}$	$\{q_1, q_2\}$
$\{q_0, q_1\}$	$\{q_2\}$
$\{q_0, q_1\}$	$\{q_1, q_2\}$

DFA